

SIMATS ENGINEERING

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5117

NAME: G. BALASUBRAMANIAN

REG.NO:192521001

DEPARTMENT: B.TECH. IT

ASSIGNMENT: C04– AT1

FACULTY NAME: DR. R. DHARANI

1) Question 1

Programiz
C Online Compiler

Programiz PRO >

main.c

Run

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 // Simple MAC generation function
7 // (Educational simulation of AH authentication)
8 unsigned int generateMAC(char message[], char key[]) {
9     unsigned int mac = 0;
10    int i;
11
12    for (i = 0; message[i] != '\0'; i++)
13        mac = mac + (message[i] * (i + 1));
14
15    for (i = 0; key[i] != '\0'; i++)
16        mac = mac + (key[i] * (i + 1));
17
18    return mac;
19 }
20
```

Output

=== IPsec Authentication Header (AH) Simulation ===

Enter IP Packet Data: 1234
Enter Authentication Key: 4568

--- Sender Side ---

Original Packet : 1234
Generated MAC : 1054

--- Receiver Side ---

Received Packet : 1234
Received MAC : 1054
Calculated MAC : 1054

Authentication Successful!
IP Packet Integrity Verified.

=== Code Execution Successful ===

Question 2

main.c	Run	Output
<pre>1 #include <stdio.h> 2 #include <string.h> 3 4 #define MAX 200 5 6 // XOR encryption/decryption 7 void encryptDecrypt(char data[], char key[]) { 8 int i; 9 int keyLength = strlen(key); 10 11 for (i = 0; data[i] != '\0'; i++) { 12 data[i] = data[i] ^ key[i % keyLength]; 13 } 14 } 15 16 // Display encrypted data in hexadecimal 17 void displayHex(char data[]) { 18 int i; 19 20 for (i = 0; data[i] != '\0'; i++) { 21 printf("%02X ", (unsigned char)data[i]);</pre>		<pre>=== IPSec ESP Simulation === Enter IP Packet Payload: HelloIP Enter Encryption Key: Secret --- Sender Side --- Original Payload : HelloIP Encrypted Payload: 1B ESP packet is transmitted... --- Receiver Side --- Decrypted Payload: H ESP Decryption Successful! Original IP Packet Payload Recovered. === Code Execution Successful ===</pre>

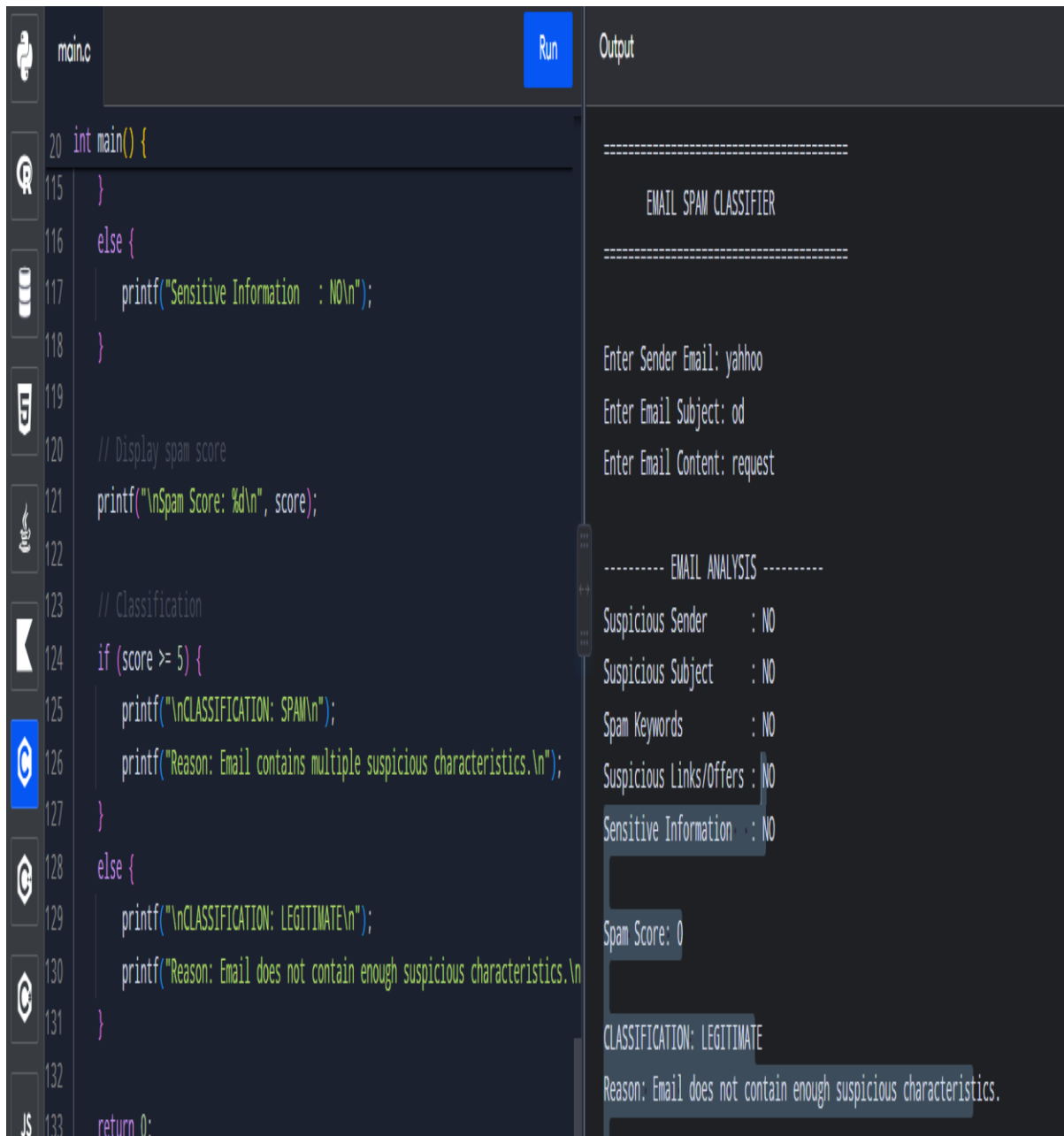
Question 3

main.c	Run	Output
<pre>1 #include <stdio.h> 2 #include <string.h> 3 4 #define MAX_SA 20 5 6 // Security Association structure 7 struct SecurityAssociation { 8 int spi; 9 char srcIP[20]; 10 char destIP[20]; 11 char protocol[10]; 12 char encryption[20]; 13 char key[30]; 14 }; 15 16 struct SecurityAssociation sa[MAX_SA]; 17 int saCount = 0; 18 19 // Create a new Security Association 20 void createSA() {</pre>		<pre>===== IPSec Security Association Manager ===== 1. Create SA 2. Lookup SA 3. Delete SA 4. Display All SAs 5. Exit Enter your choice: 1 Enter SPI: 1001 Enter Source IP: 192.168.32.16 Enter Destination IP: 192.168.16.27 Enter Protocol (AH/ESP): 56 Enter Encryption Algorithm: 123 Enter Security Key: 2 Security Association created successfully!</pre>

Question 4

main.c	Run	Output
<pre>1 #include <stdio.h> 2 #include <string.h> 3 4 long long modPow(long long base, long long exp, long long mod) { 5 long long result = 1; 6 7 while (exp > 0) { 8 if (exp % 2 == 1) 9 result = (result * base) % mod; 10 11 base = (base * base) % mod; 12 exp = exp / 2; 13 } 14 15 return result; 16 } 17 18 // Simple hash function for demonstration 19 long long hashMessage(char message[]) { 20 long long hash = 0;</pre>		<pre>Enter Email Message: hello --- Sender Side --- Original Message : hello Message Hash : 1617 Digital Signature: 2446 Encrypted Data : 1617 Email is securely transmitted... --- Receiver Side --- Received Message : hello Received Signature: 1617 Decrypted Hash : 1873 Calculated Hash : 1617 PGP Verification Failed! Message may have been modified or signature is invalid.</pre>

Question 5



The image shows a C++ IDE with a file named `main.c`. The code implements an email spam classifier. It prompts the user for sender email, subject, and content, calculates a spam score, and classifies the email as either SPAM or LEGITIMATE based on the score. The output window shows the program's execution with user input and the resulting classification.

```
main.c Run Output

20 int main() {
115 }
116 else {
117     printf("Sensitive Information : NO\n");
118 }
119
120 // Display spam score
121 printf("\nSpam Score: %d\n", score);
122
123 // Classification
124 if (score >= 5) {
125     printf("\nCLASSIFICATION: SPAM\n");
126     printf("Reason: Email contains multiple suspicious characteristics.\n");
127 }
128 else {
129     printf("\nCLASSIFICATION: LEGITIMATE\n");
130     printf("Reason: Email does not contain enough suspicious characteristics.\n");
131 }
132
133 return 0;
```

Output

```
=====
EMAIL SPAM CLASSIFIER
=====

Enter Sender Email: yahhoo
Enter Email Subject: od
Enter Email Content: request

----- EMAIL ANALYSIS -----
Suspicious Sender : NO
Suspicious Subject : NO
Spam Keywords : NO
Suspicious Links/Offers : NO
Sensitive Information : NO

Spam Score: 0

CLASSIFICATION: LEGITIMATE
Reason: Email does not contain enough suspicious characteristics.
```